# Remote SITS

Omkar Zarikar
Nathan Varner
Darren Davis
Tenghoit Kouch

## Problem

After the successful launch of the SITS program, Director Coderson has directed us to further develop the software. The company wants to integrate RestClient to allow the users to remotely access the software. Each user can register a robot through the web into a tournament that the user can pick from. The company also wants to expand the reach of our product and wants to allow for a user to input actions for the game instead of creating their own robot's algorithm.

## Diagrams

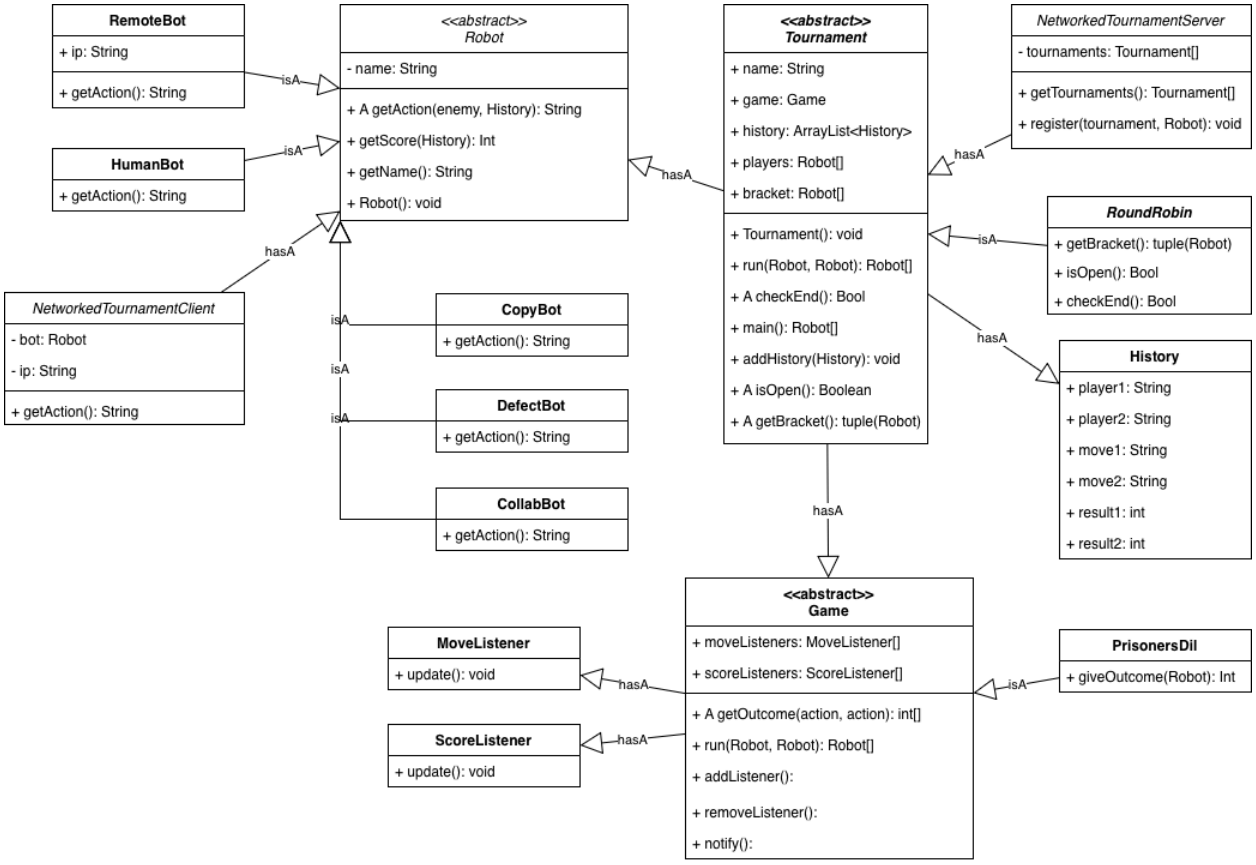


***Figure 1*** *shows our full UML diagram for the remote SITS system.*

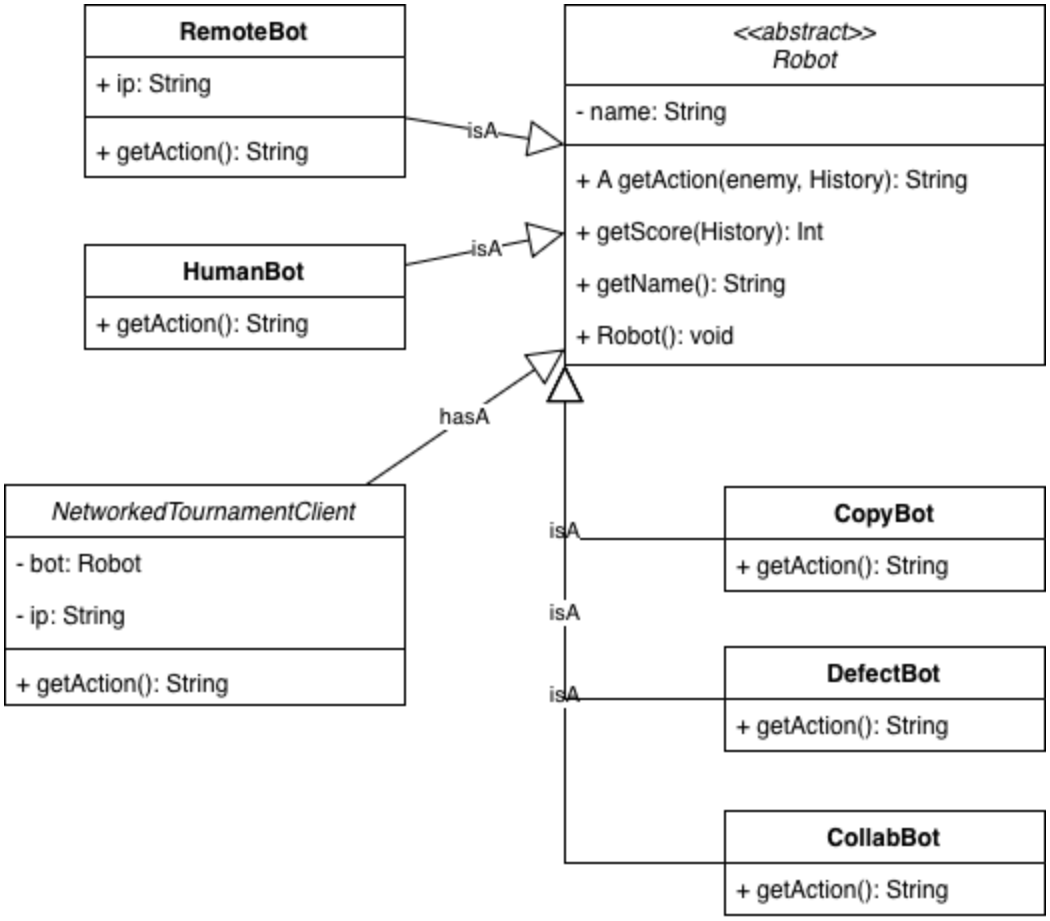


***Figure 2*** *shows the design for the NetworkedTournamentClient.*

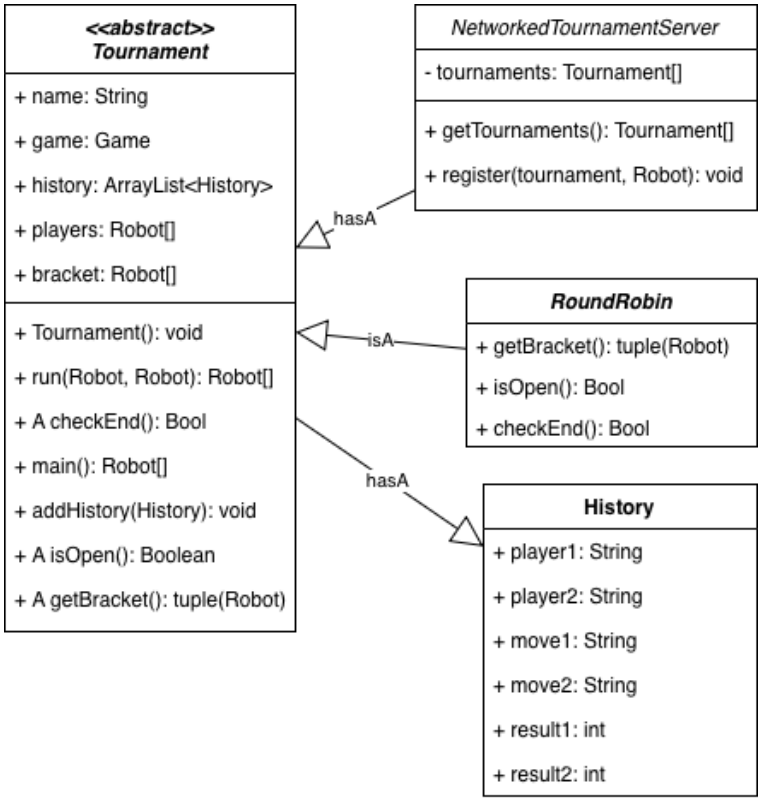


**Figure 3** shows the UML for the NetworkedTournamentServer.

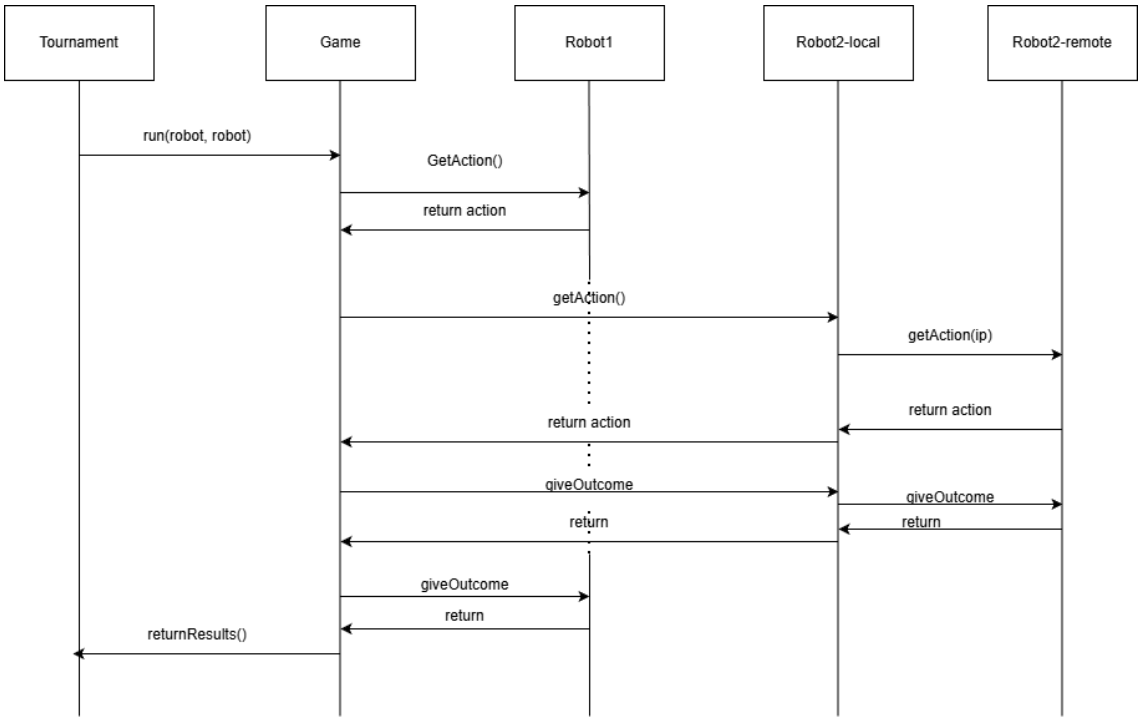


**Figure 4:** Sequence diagram of Remote Action

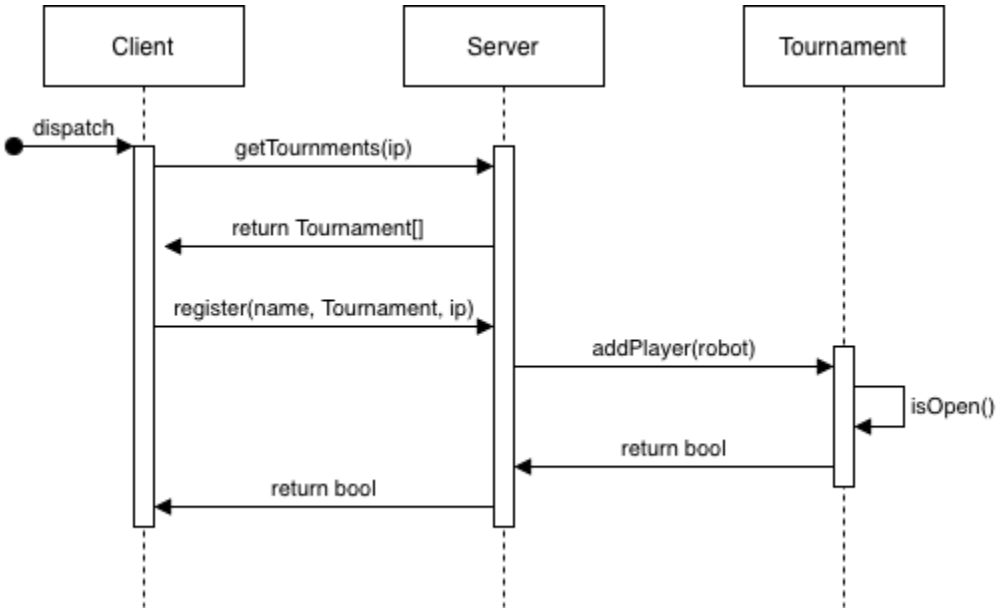


**Figure 5:** Sequence diagram of Registration

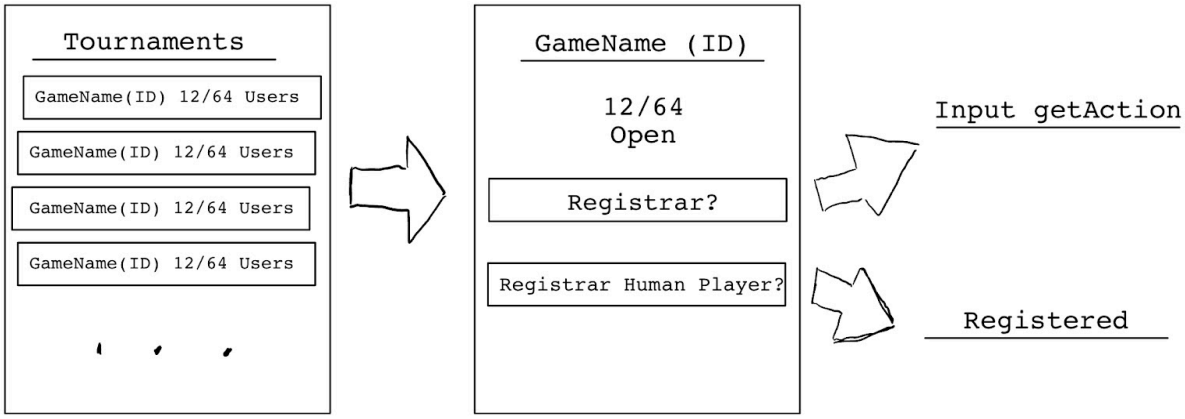


**Figure 6:** Rough Html Design Idea

The Remote robot differs from a local Robot because it doesn't run getactions() itself inside the tournament process. Instead, it holds the client's IP address and uses it to reach back to the original client machine whenever it needs to decide action.

# Human Participant

The player can choose whether they will register their own robot implementation or decide to choose their own as shown in figure 6. The human player will be given a different subclass of Robot which will take a scanner call as its getAction, allowing the user to simply input their action each turn.

# Remote Participant

If the player chooses to remotely implement their own code they will be prompted to input their implementation of the getAction method. This implementation will be pulled from the web remotely by another new subclass of Robot which will return the string given to the client by looking for the ip of that client.

When a remote participant registers for a particular tournament, our server essentially creates a “proxy” robot that acts as the representative for the participant in the tournament. This subclass of Robot, RemoteBot, has some unique properties. At creation, it stores the ip address of the remote participant and its getAction() simply makes a request to the stored ip address and passes along the result. This is great because neither Game nor Tournament need to deal with any networking issues. Since as far as they can tell, RemoteBot is just another Robot and thus no modification is needed for the original design.

# Server

Our server is the object we use in order to register players into an available tournament. When a Client decides to register for a tournament we add a player to the list of robots in the tournament decided upon by the client which is given via string describing the name of the tournament they wish to register for in the register function. Then assuming the tournament is available to join we can now add a human or remote robot to the players list depending on what the client chooses. The tournament is then eventually run when that tournament is no longer open/registration phase has ended.

# Client

A client has a robot which is either a human bot situation or a created robot and registers with a robot name and tournament name in the server. We then check the value of isOpen for that tournament and if TRUE then we send a confirmation message and add the human bot or the remote participant to the tournament players list. We do also have an ip address in clients to allow for our remote participant system to work.